

Search Under a Disorder Budget

A note on preprocessing, repeated queries, and learned structure

Jonathan R. Landers October 7, 2026

The paper *Disorder Potential and the Cost of Search* asks what a single search can gain from knowing how far an array lies from sorted order. Its answer has a square-root term: a potential budget of size E permits a key to hide among roughly $\sqrt{E}$ positions. Repeated searches change the question. Comparisons spent once on the fixed array can create an index that later targets share. Here E limits the input's disorder; a separate budget B pays for learning its structure.

The one-query starting point. Let $A = (a_1, \dots, a_n)$ contain distinct keys, and let $x_i = \text{rank}_A(a_i)$. The rank word x is a vertex of the permutohedron; sorted order is $(1, \dots, n)$. The paper uses

$$V(A) = \frac{1}{2} \sum_{i=1}^n (x_i - i)^2, \quad d(A) = \max_i |x_i - i|, \quad V(A) \leq E.$$

Set

$$D_E = \min \left\{ n - 1, \left\lfloor \frac{\sqrt{1 + 8E} - 1}{2} \right\rfloor \right\}, \quad R_E = D_E + 1.$$

The displacement lemma gives $d(A) \leq D_E$: if one displacement has magnitude d , the others have total absolute displacement at least d , because $\sum_i (x_i - i) = 0$. Since displacements are integers, $2V(A) \geq d^2 + d$. Thus $R_E = \Theta(1 + \min\{n, \sqrt{E}\})$.

A target query compares y with entries a_i and returns its index or absence. With no preprocessing, the paper proves a worst-case cost $\Theta(\log n + R_E)$. For the upper bound, a midpoint comparison narrows the possible target positions up to a margin of $2D_E$; when only $O(D_E + 1)$ positions remain, scan them. The lower bound moves a new minimum to position j : the resulting rank word $(2, \dots, j, 1, j + 1, \dots, n)$ has potential $j(j - 1)/2$, so every $j \leq R_E$ is a possible hiding place.

What preprocessing may buy. Now keep the same array for $q \geq 1$ target searches. Before the first target, preprocessing may compare array keys and retain an $O(n)$ -word index. Count those comparisons and all later target-to-entry comparisons; the promise $V(A) \leq E$ remains free, as in the paper. The optimization is

$$T_q^*(n, E) = \inf_{P, S} \sup_{\substack{A: V(A) \leq E \\ y_1, \dots, y_q}} \left[C_P(A) + \sum_{t=1}^q C_S(y_t \mid A, P, \mathcal{H}_{t-1}) \right],$$

over correct methods using $O(n)$ storage; $\mathcal{H}_{t-1}$ is the retained search history. A useful accounting form is

$$\text{total cost} \leq B + O(q(\log n + W)), \tag{1}$$

whenever B preprocessing comparisons produce a verified search structure with per-query residual work at most W beyond binary-search depth. The structure and its verification determine W ; a prediction alone does not.

Theorem 1 (Preprocessing bound). *For $n \geq 2$, $E \geq 0$, and $q \geq 1$,*

$$\boxed{T_q^*(n, E) = O(q \log n + \min\{qR_E, n \log(R_E + 1)\})}. \tag{2}$$

The minimum compares two concrete choices of preprocessing budget.

Proof of the bound. One choice is $B = 0$: use the paper’s bounded-displacement search independently for each target. This costs $O(q \log n + qR_E)$.

For the other choice, construct a sorted index of the array positions. The displacement bound says that the key of rank k occurs by position $k + D_E$. Start a min-heap with positions $1, \dots, \min\{n, D_E + 1\}$. Repeatedly remove its minimum and insert the next unread position, if any. Before the k th removal, the heap contains every unread position among the first $\min\{n, k + D_E\}$ positions. In particular it contains rank k , whose position is at most $k + D_E$; the extracted key is therefore rank k . A heap of at most $D_E + 1 = R_E$ entries costs $O(n \log(R_E + 1))$ comparisons and yields an $O(n)$ -word array of positions in key order. Ordinary binary search on that index then costs $O(\log n)$ comparisons per target. Choosing the cheaper strategy proves (2). $\square$

The crossover is $qR_E \asymp n \log(R_E + 1)$, up to constant factors. For small q , the original search is economical; for enough queries, paying once for the index removes the recurring displacement scan. When $E = 0$, the promise already gives sorted order. When $R_E = n$, the index construction has the familiar $O(n \log n)$ cost.

Why remembered search outcomes are weaker. Suppose there is no array-to-array preprocessing and memory retains only previous target-to-entry signs. Even this persistent memory cannot remove the qR_E term in the worst case. Run the algorithm on a sorted array $a_1 < \dots < a_n$ with increasing absent targets $y_1 < \dots < y_q < a_1$. In round t , for each $j \leq R_E$, compare this history with the fixed alternative array that replaces a_j by y_t . That alternative has potential $j(j-1)/2 \leq E$. Every earlier target is below y_t , so all earlier comparison signs agree with those on the sorted array. Unless round t queries position j , its signs agree as well, yet its correct answer is j rather than absence. Every one of the first R_E positions must therefore be queried in every round: $\Omega(qR_E)$ target comparisons. The argument concerns sign-only search histories; the sorted-index strategy uses the stronger array comparisons allowed during preprocessing.

Where a learned certificate fits. A learned rule can propose a family of likely key locations, a small displacement limit, or a partial index. Its cost enters through B , including the comparisons needed to *verify* a guarantee usable by search. For example, if preprocessing certifies $d(A) \leq D'$ after B comparisons, the paper’s search argument gives

$$\text{total cost} \leq B + O(q \log n + q(D' + 1)). \quad (3)$$

This is an instance-dependent improvement when the saved scan work amortizes B . For example, a prediction that A is sorted can be checked with $n - 1$ adjacent comparisons; on success $D' = 0$. On failure, use the baseline. A certificate of $D' < D_E$ is not guaranteed over every array with $V(A) \leq E$: some allowed arrays attain $d(A) = D_E$. The unconditional result is (2). A sharper worst-case guarantee from learning needs a stated, verified restriction on the arrays or their possible target locations.

Context. The potential, displacement lemma, and single-search theorem are from Landers, *Disorder Potential and the Cost of Search*, version 2. The bounded-displacement search connection is also discussed by Disser and Kratsch, *Robust and Adaptive Search*, STACS 2017, doi:10.4230/LIPIcs.STACS.2017.26.